Chapter 12: Deep Learning with Neural Networks Part I

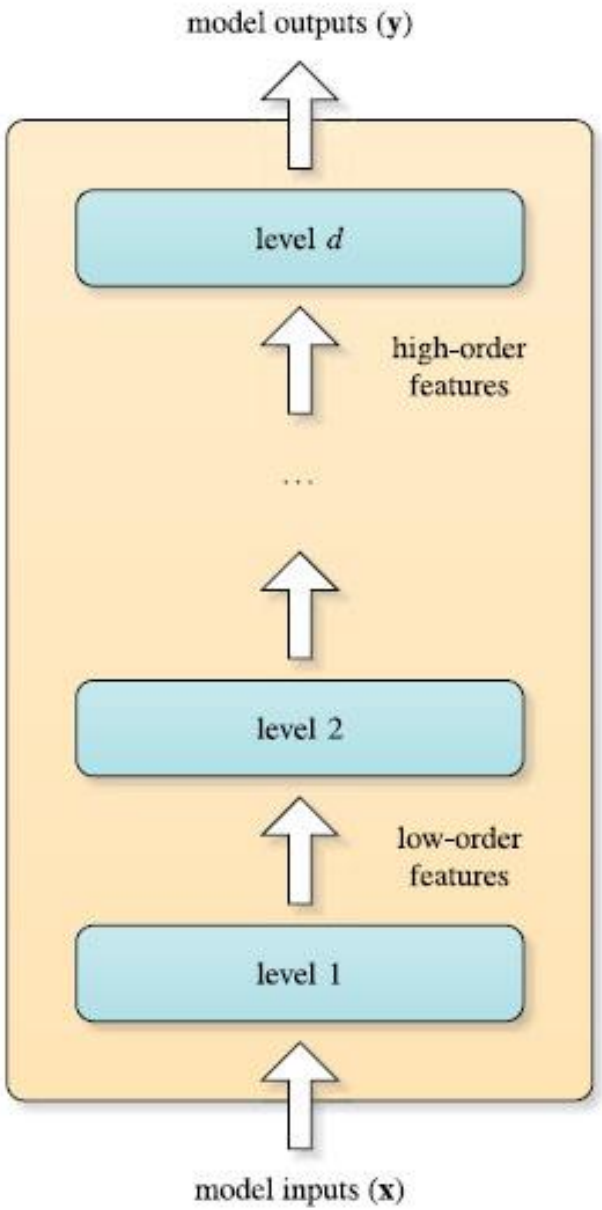
Assoc. Prof. Dr. Duong Tuan Anh
Faculty of Computer Science and Engineering,
HCMC Univ. of Technology
12/2016

Outline

- Deep Learning
- Deep Belief Neural Networks
- Convolutional Neural Networks
- Tools for Deep Neural Networks
- Applications of Deep Neural Networks
- Conclusions
- Appendix

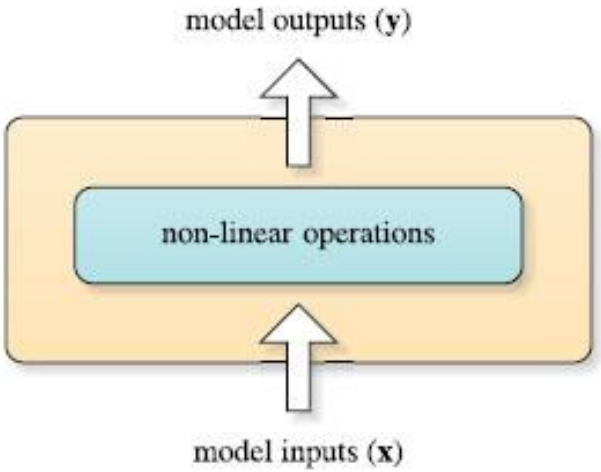
1. Deep Learning

- Deep learning is a class of machine learning techniques, where ***many layers*** of information-processing stages in ***hierarchical architectures*** are exploited for pattern classification and for *feature /representation learning*.
- Deep architecture model consists of many layers of composition of *non-linear operations* in its outputs.
- The idea is to have ***feature detector units*** at each layer that gradually extract and refine more sophisticated and invariant features from the original raw input signals.
- Lower layers aim at extracting *simple features* that are then clamped into higher layers which in turn detect more *complex features*.
- In contrast, ***shallow models*** (e.g. one hidden layer NNs) present very few layers of composition that basically map the original input features into a problem-specific feature space.



deep architecture

Figure 12.1 illustrates the main architecture differences between deep and shallow models.



shallow architecture

Deep learning

- Deep architectures can be *exponentially* more efficient than shallow ones.
- Deep learning is in the intersections among the areas of *neural networks, graphical modeling, optimization, pattern recognition* and *signal processing*.
- This chapter focuses on deep learning with neural networks through three kinds of Deep Neural Networks:
  - Deep Belief Neural Networks
  - Convolutional Neural Networks
  - Long Short Term Memory Networks
- Any neural network with more than two hidden layers is deep.

Deep neural network

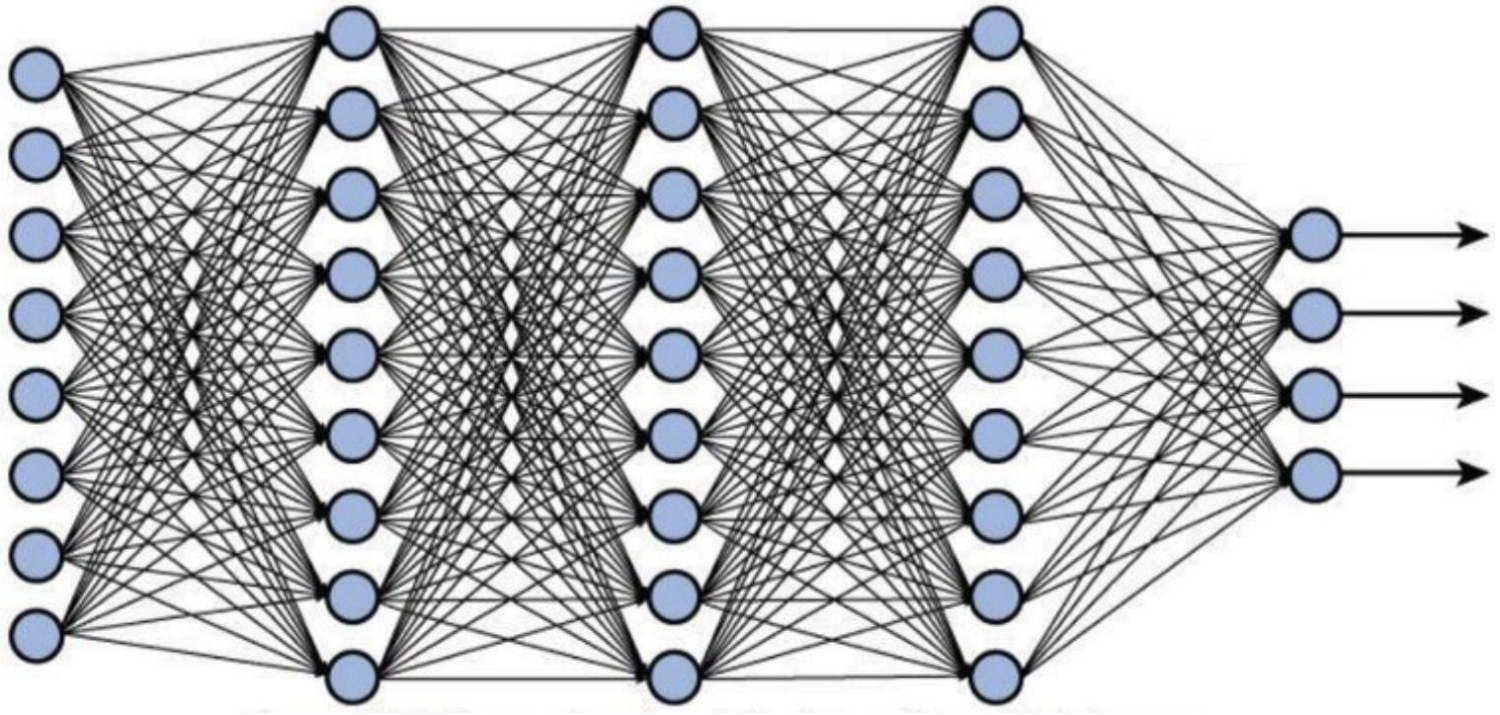


Figure 12.2 A deep neural network with 3 hidden layers

History of deep learning

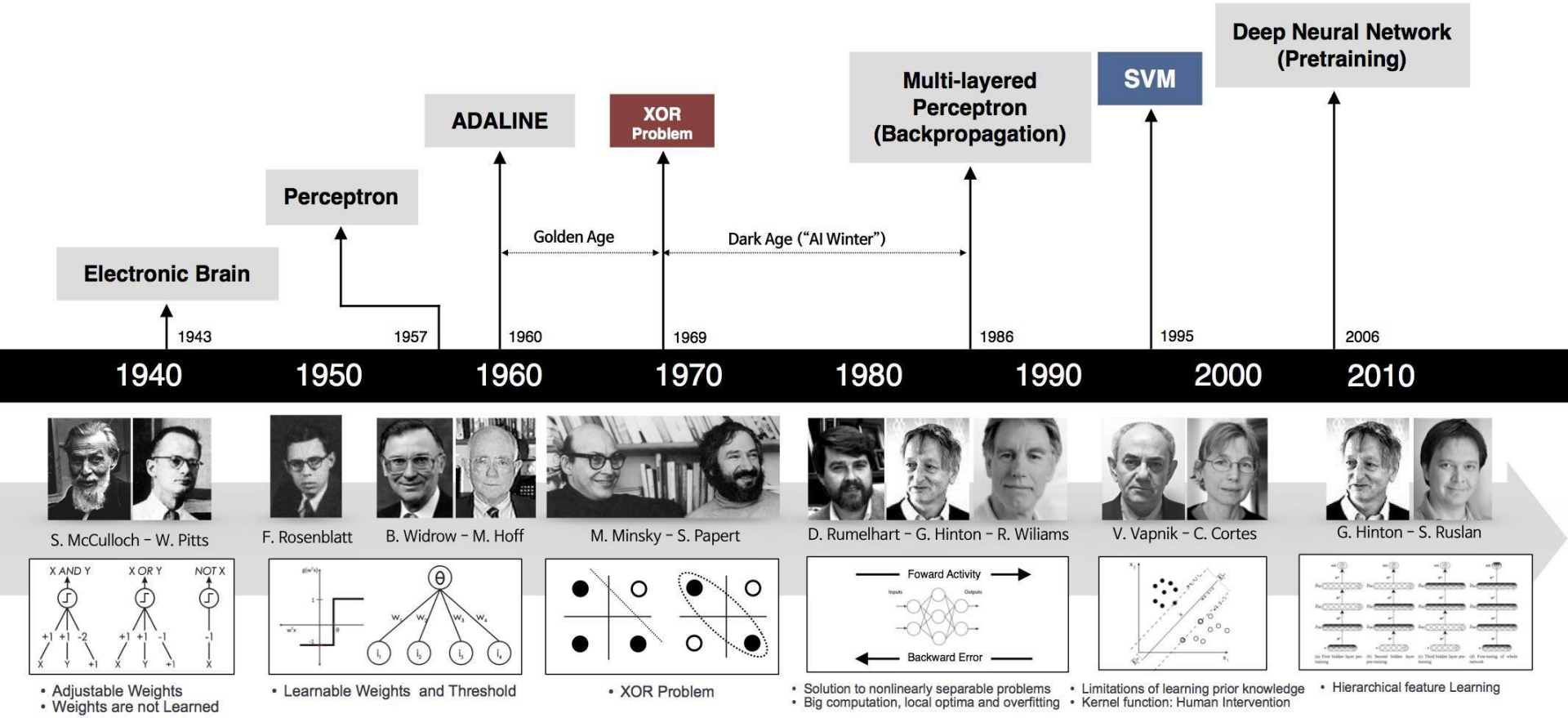


Figure 12.3

2. Deep Belief Neural Networks

- Deep Belief Neural Network (DBNN) is one of the first application of deep learning.
- Hinton (2007) describes DBNNs as “probabilistic generative models that are composed of multiple layers of stochastic, latent variables”.
- *Probabilistic*- DBNNs are used to classify, and their output is the probability that an input belongs to each class.
- *Generative* – DBNNs can produce randomly created values for the input values.
- Multiple layers.
- Stochastic, *latent* variables- DBNNs are made up of Boltzmann machines that produce random values that can not be directly observed (latent).

Differences between DBNN and FFNN

- Input to a DBNN must be binary; input to a feed-forward neural network (FFNN) is a real number.
- The output from a DBNN is the class to which the input belongs; the output from a FFNN can be a class (classfication) or a numeric prediction.
- DBNNs can ***generate*** plausible input based on a given outcome. FFNNs cannot perform like the DBNNs.

Note: ***Generative models*** provide a joint probability distribution over observable data and labels, facilitating the estimation of *P*(*Observation*| *Label*) as well as *P*(*Label*| *Observation*), while ***discriminative models*** are limited to the latter, *P*(*Label*| *Observation*)

DBNN architecture

- DBNNs are composed of several layers of *Restricted Boltzmann Machines.*
- Input provided to the DBNN passes through a series of *stacked RBMs* that make up the layers of the network.
- Though RBMs are unsupervised, we want the resulting DBNN to be supervised.

Boltzmann Machines

- A Boltzmann machine is a *fully connected*, two layer neural network. These layers are called *visual* and *hidden* layers.
- The visual layer is similar to input layer and the hidden layer is similar to output layer in feed-forward neural network.
- The Boltzmann machine has no hidden layer.

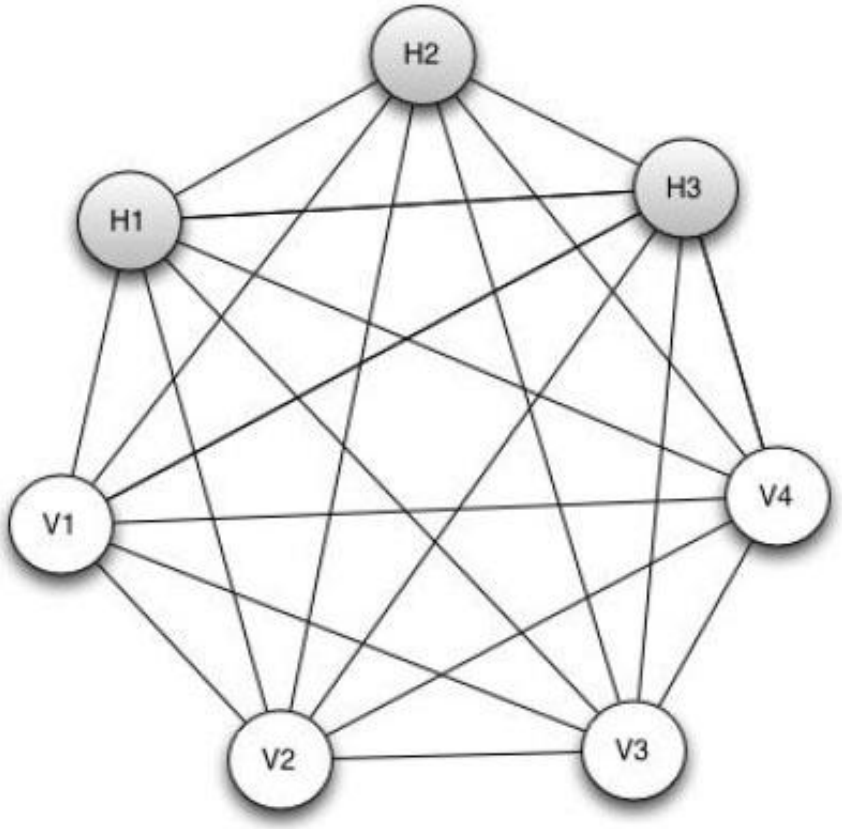


Figure 12.4

A Boltzmann machine is fully connected because every neuron has a connection to every other neuron.

Restricted Boltzmann Machines (RBMs)

- RBM is not fully connected. All hidden neurons are connected each visual neuron. However, there is no connections among the hidden neurons nor are the connections among the visual neurons.
- A Boltzmann machine's neurons acquire only binary states, either 0 or 1.

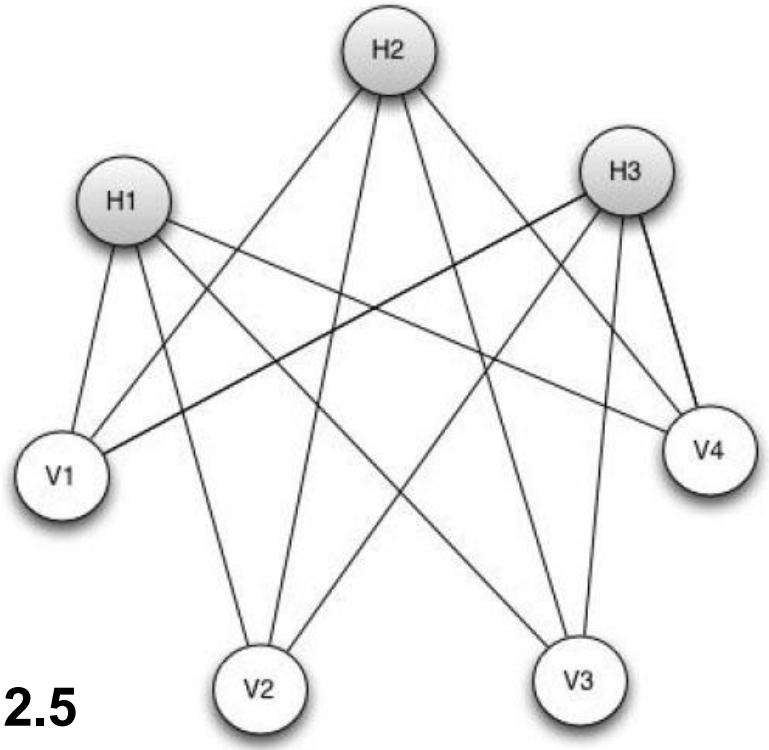
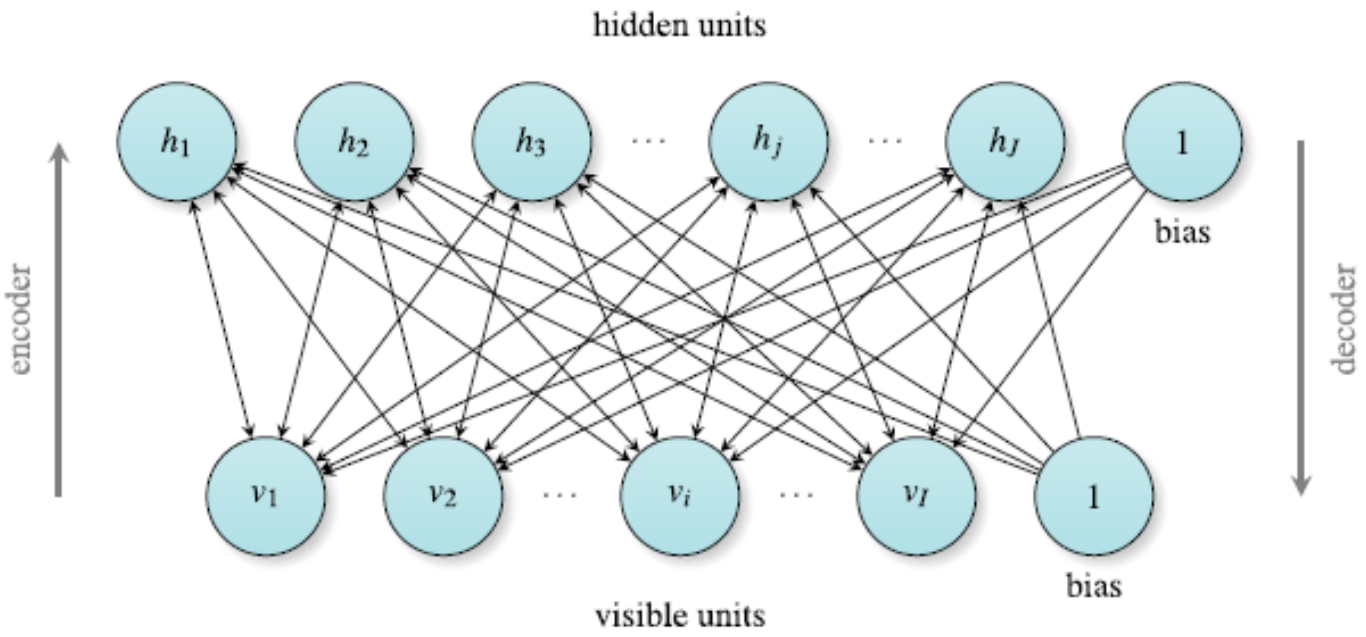


Figure 12.5

Boltzmann machines are called a ***generative*** model. A Boltzmann machine does not generate constant output. The values presented to the visible neurons of a Boltzmann machine, when considered with the weights, specify a *probability* that the hidden neurons will assume a value of 1, as opposed to 0.

- RBMs follow the ***encoder-decoder*** paradigm. In this paradigm an ***encoder*** transform the input into a feature vector representation from which a ***decoder*** can reconstruct the original input. In RBMs, both the encoded representation and the decoded reconstruction are stochastic by nature.
- An RBM is an **energy-based generative model**. It consist of a layer of I binary visible units (observable variables), $\boldsymbol{v} = [v_1, v_2, \ldots, v_I]$ where $v_i \in \{0, 1\}$, and a layer of J binary hidden units, $\boldsymbol{h} = [h_1, h_2, \ldots, h_J]$ where $h_j \in \{0, 1\}$, with bidirectional weighted connections.

Figure 12.6



Energy function of RBM

- The stability of a RBM can be described by an **energy function**. Given an observed state, the energy of the **joint** configuration of the visible and hidden units (**v**,**h**) is given by (12.1)

$$E(\mathbf{v},\mathbf{h}) = -\mathbf{c}\mathbf{v}^\top - \mathbf{b}\mathbf{h}^\top - \mathbf{h}\mathbf{W}\mathbf{v}^\top$$
$$= -\sum_{i=1}^{I} c_i v_i - \sum_{j=1}^{J} b_j h_j - \sum_{j=1}^{J}\sum_{i=1}^{I} W_{ji} v_i h_j \,, \qquad (12.1)$$

where $\boldsymbol{W}$ is a $J \times I$ matrix containing the RBM connection weights, $\boldsymbol{c} = [c_1, c_2, \ldots, c_I]$ is the bias of the visible units and $\boldsymbol{b} = [b_1, b_2, \ldots, b_J]$ the bias of the hidden units.

RBM probability

- The RBM assigns a probability for each configuration (***v***, ***h***), using (12.2)

$$p(\mathbf{v},\mathbf{h}) = \frac{e^{-E(\mathbf{v},\mathbf{h})}}{Z}, \qquad (12.2)$$

where ***Z*** is a *normalization constant* called *partition function* by analogy with physical systems, which is obtained by summing up the energy of all possible (***v***, ***h***) configurations.

$$Z = \sum_{\mathbf{v},\mathbf{h}} e^{-E(\mathbf{v},\mathbf{h})}. \qquad (12.3)$$

RBM probability (cont.)

- Since there are no connections between any two units within the same layer, given a particular random input configuration, ***v***, all hidden units are independent of each other and the probability of ***h*** given ***v*** become:

$$p(\mathbf{h} \mid \mathbf{v}) = \prod_j p(h_j = 1 \mid \mathbf{v}) , \qquad (12.4)$$

$$p(h_j = 1 \mid \mathbf{v}) = \sigma(b_j + \sum_{i=1}^{I} v_i W_{ji}) . \qquad (12.5)$$

σ is *sigmoid* function.

For implementation purposes, h_j is set to 1 when $p(h_j = 1|\boldsymbol{v})$ is greater than a random number (between 0 and 1) and 0 otherwise.

Probability of visible vector: p(v)

- Similarly given a specific hidden state, the probability of ***v*** given ***h*** is obtained by:

$$p(\mathbf{v} \mid \mathbf{h}) = \prod_i p(v_i = 1 \mid \mathbf{h}) , \qquad (12.6)$$

$$p(v_i = 1 \mid \mathbf{h}) = \sigma(c_i + \sum_{j=1}^{J} h_j W_{ji}) . \qquad (12.7)$$

When using (12.7) in order to reconstruct the input vector, it is vital to force the hidden states to be binary.
The **marginal probability** assigned to a *visible vector*, ***v***, is given by:

$$p(\mathbf{v}) = \sum_{\mathbf{h}} p(\mathbf{v}, \mathbf{h}) = \frac{1}{Z} \sum_{\mathbf{h}} e^{-E(\mathbf{v}, \mathbf{h})} . \qquad (12.8)$$

Contrastive Divergence Algorithm

- To train the Restricted Boltzmann Machine, Hinton proposed a faster learning procedure: the Contrastive Divergence (CD-k) algorithm.
- This algorithm is simple and effective.
- In CD-k, we obtain the following update rule for the weights of the network:

$$\Delta W_{ji} = \eta \left(\overbrace{\langle v_i h_j \rangle_0}^{\text{positive phase}} - \underbrace{\langle v_i h_j \rangle_k}_{\text{negative phase}} \right) \qquad (12.9)$$

where η represents the *learning rate,*
$\langle v_i h_j \rangle$ means sampling $p(h|v)$.

CD-k Algorithm (cont.)

- Similarly, we obtain the following rule for the update the bias of the network:

$$\Delta b_j = \eta(\langle h_j \rangle_0 - \langle h_j \rangle_k) \qquad (12.10)$$

$$\Delta c_i = \eta(\langle v_i \rangle_0 - \langle v_i \rangle_k) \qquad (12.11)$$

In CD-k algorithm, it uses the probabililies associated to the visible units for computing the state of the hidden units.

The main steps of the CD-k algorithm (for training a RBM) is as follows.

CD-k Algorithm

1. $\mathbf{v}^{(0)}$:= **x** /* **x** is an input vector of the training dataset
2. Computing the binary states of the hidden units, $\mathbf{h}^{(0)}$, using $\mathbf{v}^{(0)}$ and Eq. 12.5.
3. **for** n := 1 **to** k **do**
4. Compute the "reconstruction" states for the visible unit, $\mathbf{v}^{(n)}$,using $\mathbf{h}^{(n-1)}$ and Eq. 12.7.
5. Compute the binary features (states) for the hidden unit, $\mathbf{h}^{(n)}$, using $\mathbf{v}^{(n)}$ and Eq. 12.5.
6. **endfor**
7. Update the weights and biases, using Eq. 12.9 , Eq.12.10. and Eq.12.11

Properties of CD-k

- The CD-k is a rough approximation algorithm for training RBM, however it has been successfully applied to many significant applications.
- CD-k algorithm can create good *generative models* from the training dataset.
- The magnitude of the CD-k gradient estimate may not be correct, but its direction tends to be accurate.
- In particular, CD-1 provides a low variance, fast and reasonable approximation to the log-likelihood gradient.

Deep Belief Network Architecture

Since a DBN aims to maximize the likelihood of the training data, *the training process starts by the low-level RBM* that receives the DBN inputs, and progressively moves up in the hierarchy, until finally the RBM in top layer, containing the DBN outputs, is trained.

Figure 12.7

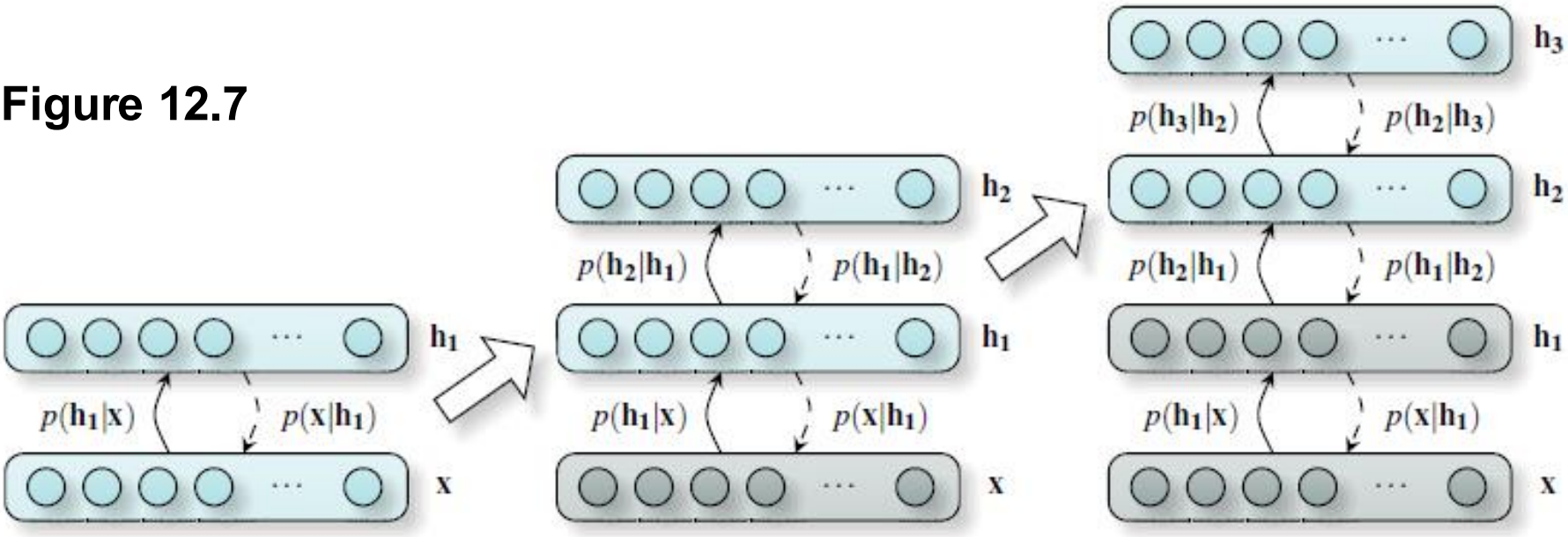


Figure 12.7 shows the training process of a Deep Belief Network with one layer ***x***, and three hidden layers h_1, h_2, h_3.

Training a DBNN

- The training process of DBNNs, also called **pre-training,** is *unsupervised* by nature, allowing the system to learn non-linear complex mapping functions directly from data.
- In order to perform classification, *additional layer* is added at the end of the last hidden layer. This layer is also connected to the last hidden layer with corresponding weights.
- By adding an *additional layer*, the resulting network is **fine-tuned** by using the Back-Propagation algorithm.
- Each DBNN layer is trained ind*ependently* during the unsupervised part. It is possible to train the DBNN layers *concurrently* (with threads) during unsupervised part.
- During the *supervised* part (with BP algorithm), only training data with labels are used.

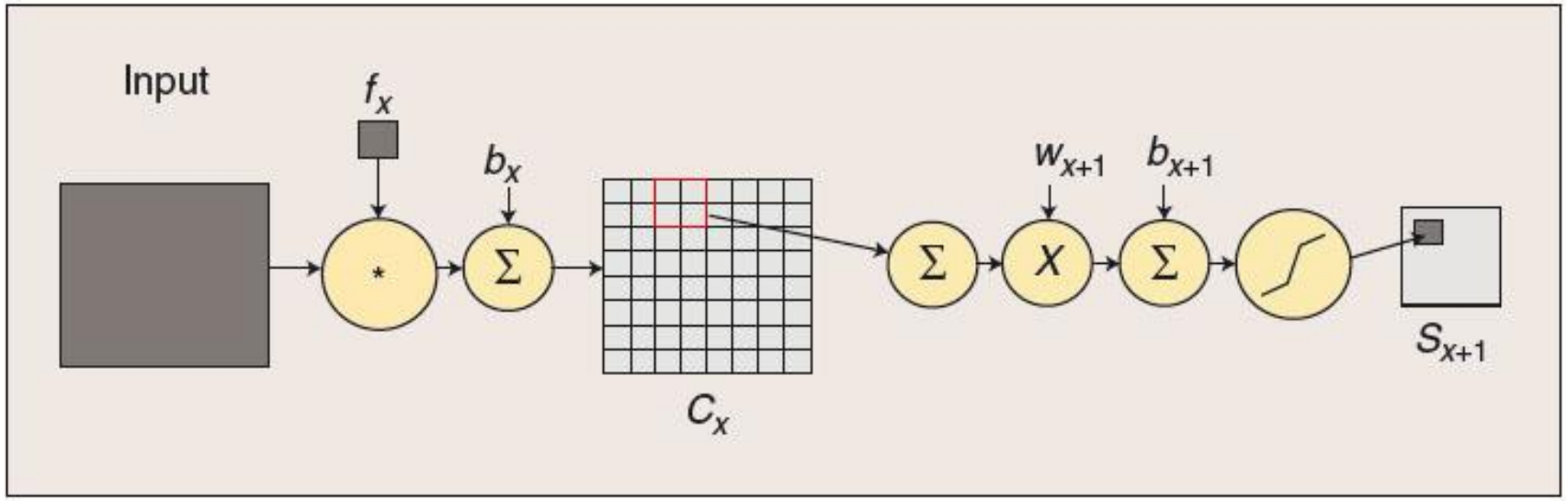
Parameters of DBNs

- DBN has the following parameters:
  - The number of layers
  - The number of units in each hidden layer
  - The learning rates
  - The number of epochs in training DBNs
  - The initialization of weights and bias.
- The best parameter values can be discovered through experiments or using meta-heuristics.

3. Convolutional Neural Networks

- CNNs are a family of multi-layer neural networks particular designed for use on two-dimensional data, such as, images and videos.
- CNNs take advantage of spatial structure of the images.
- In CNNs, small portions of the image (called *local receptsive field*) are treated as inputs to the lowest layer of the hierarchical structure.
- Information propagates through different layers of the network where at each layer *digital filtering* is applied in order to obtain salient features of the data observed.
- The following two layer types comprise the CNNs:
  - Convolution layer
  - Subsampling layer

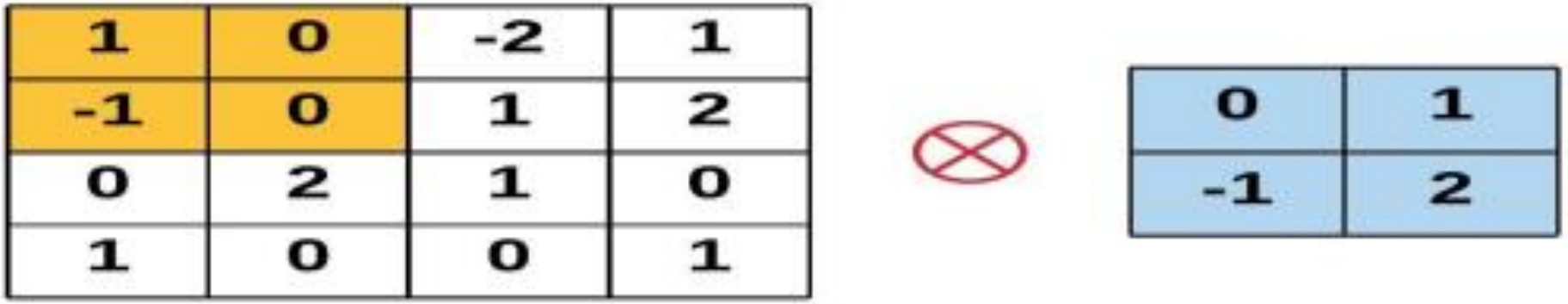
Figure 12.8: The convolution and subsampling process



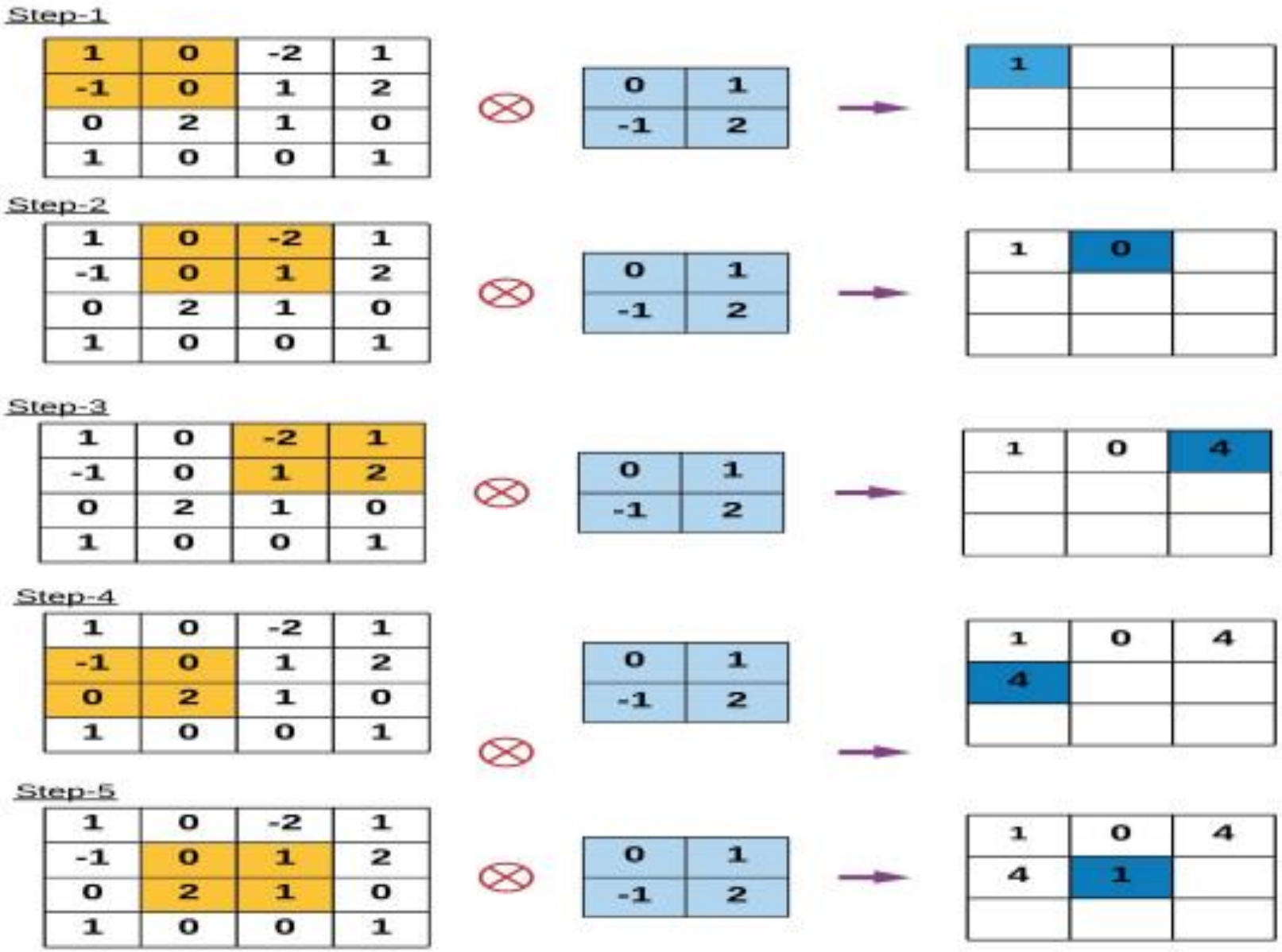
- ***Convolution*** process consists of convolving an input (image) with a trainable filter f_x, then adding a trainable bias, b_x to produce the convolution layer C_x.
- ***Subsampling*** consists of summing a neighborhood, weighting by scalar w_{x+1}, adding trainable bias b_{x+1} and passing through a sigmoid function to produce a smaller feature map S_{x+1}.

Convolution operation

The filter



Hình 12.9


Step-1
1 0 -2 1
-1 0 1 2
0 2 1 0
1 0 0 1
0 1
-1 2
1
Step-2
1 0 -2 1
-1 0 1 2
0 2 1 0
1 0 0 1
0 1
-1 2
1 0
Step-3
1 0 -2 1
-1 0 1 2
0 2 1 0
1 0 0 1
0 1
-1 2
1 0 4
Step-4
1 0 -2 1
-1 0 1 2
0 2 1 0
1 0 0 1
0 1
-1 2
1 0 4
4
Step-5
1 0 -2 1
-1 0 1 2
0 2 1 0
1 0 0 1
0 1
-1 2
1 0 4
4 1

Hình 12.10

Result

1	0	4
4	1	1
1	1	2

The final feature map after the complete convolution operation

Convolution and Subsampling

- The input image is convolved with a set of *N* small *filters* whose coefficients are either trained or predetermined.
- The first (lowest) layer of the network consists of "*feature maps*" which are the result of the convolution process, with an additive bias and a compression or normalization of the features.
- This initial stage is followed by a subsambling (e.g. a 2×2 averaging operation) that further reduces the dimensionality and offers some robustness to spatial shifts.
- Then the subsampled *feature map* receives a weighting and trainable bias and propagates through an activation function.

- These outputs form a new feature map that is then passed through another sequence of convolution, subsampling and activation function flow.
- This process can be repeated an arbitrary number of times.

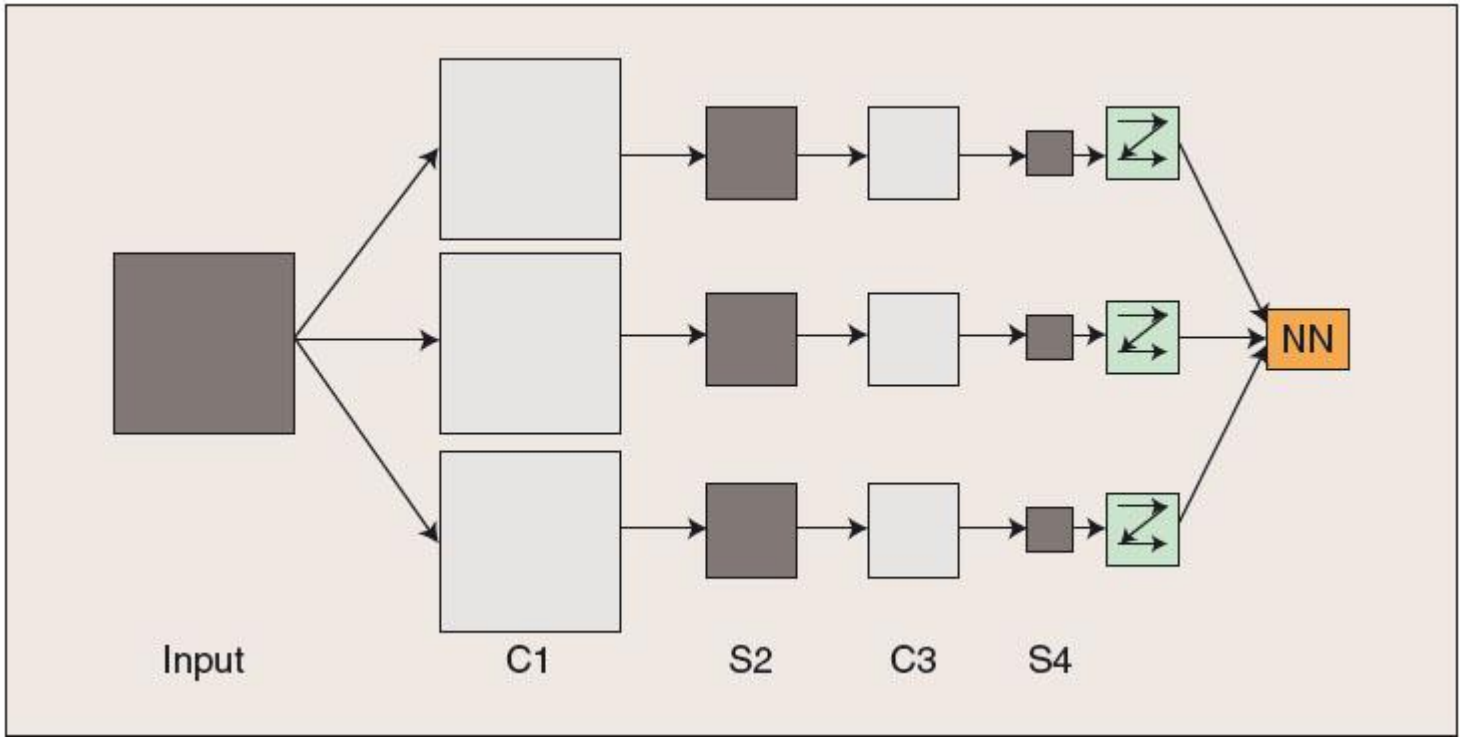


Fig. 12.11

Subsampling layer = pooling layer

Example of Convolution neural network

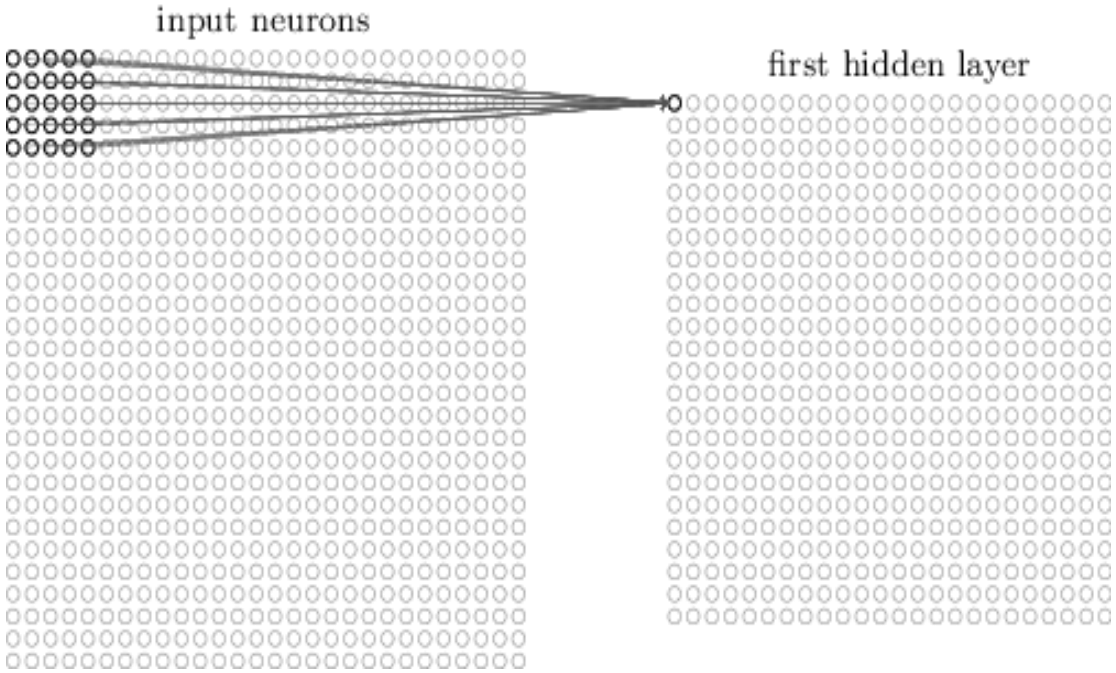
- In Figure 12.11, the input image is convolved with three trainable ***filters*** and biases to produce three ***feature maps*** at C1 level.
- Each group of four pixels in the feature maps are added, weighted, combined with a bias, and passed through a sigmoid function to produce the three feature maps at S2.
- The three feature maps at S2 are again filtered to produce the C3 level. The hierarchy then produced S4 in a manner analogous to S2.
- Finally these pixel values are ***rasterized*** and presented as a single vector input to the “conventional” neural network at the output.

Dense layer

- Finally, at the final stage of the process, the activation outputs are forwarded to a conventional feed-forward neural network that produces the final output of the system (this layer is also called *dense layer*).
- The intimate relationship between the layers and spatial information in CNNs makes them well suited for image processing.
- Summary:
  - Subsampling layers can *down-sample* the image and remove detail.
  - Convolution layers *detect features* in any part of the image field.

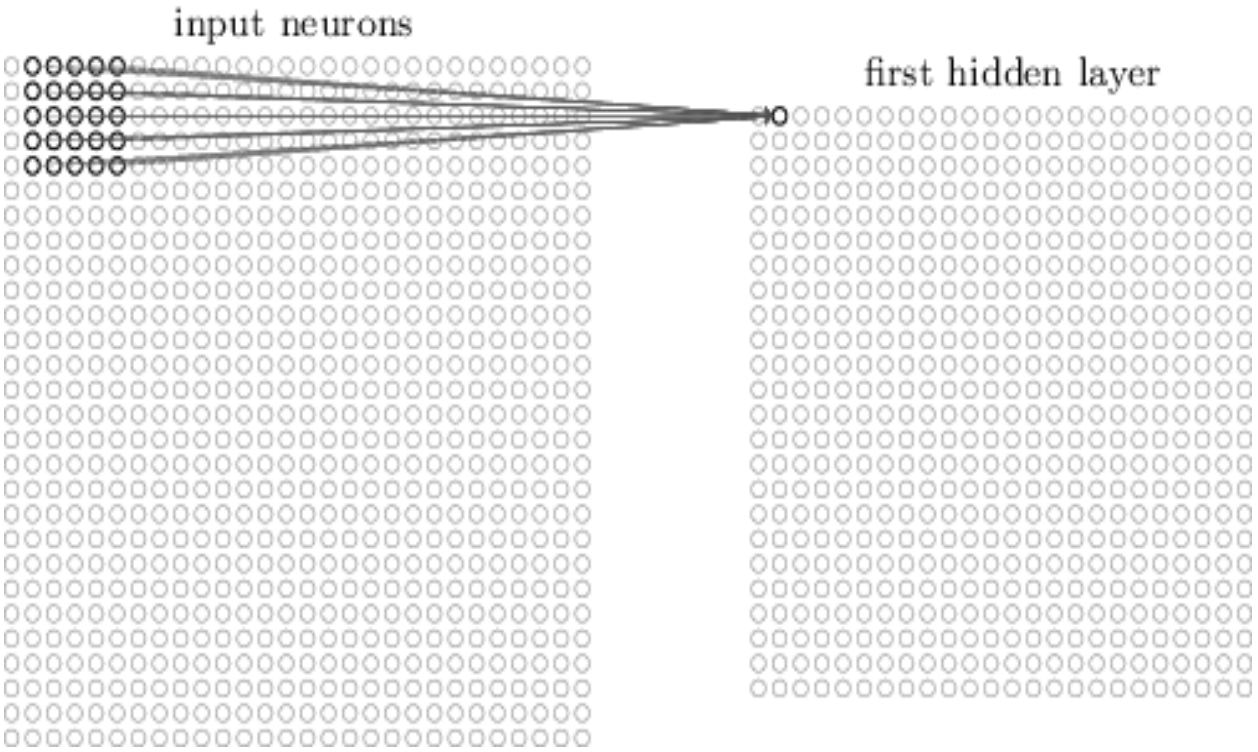
Local receptive field

Figure 12.12



- Each neuron in the first hidden layer will be connected a small region of the input neuron, e.g. a 5×5 region, corresponding to 25 input pixels.
- That region in the input image is called ***local receptive field***.
- The we ***slide*** the local receptive field across the entire input image.

Figure 12.13

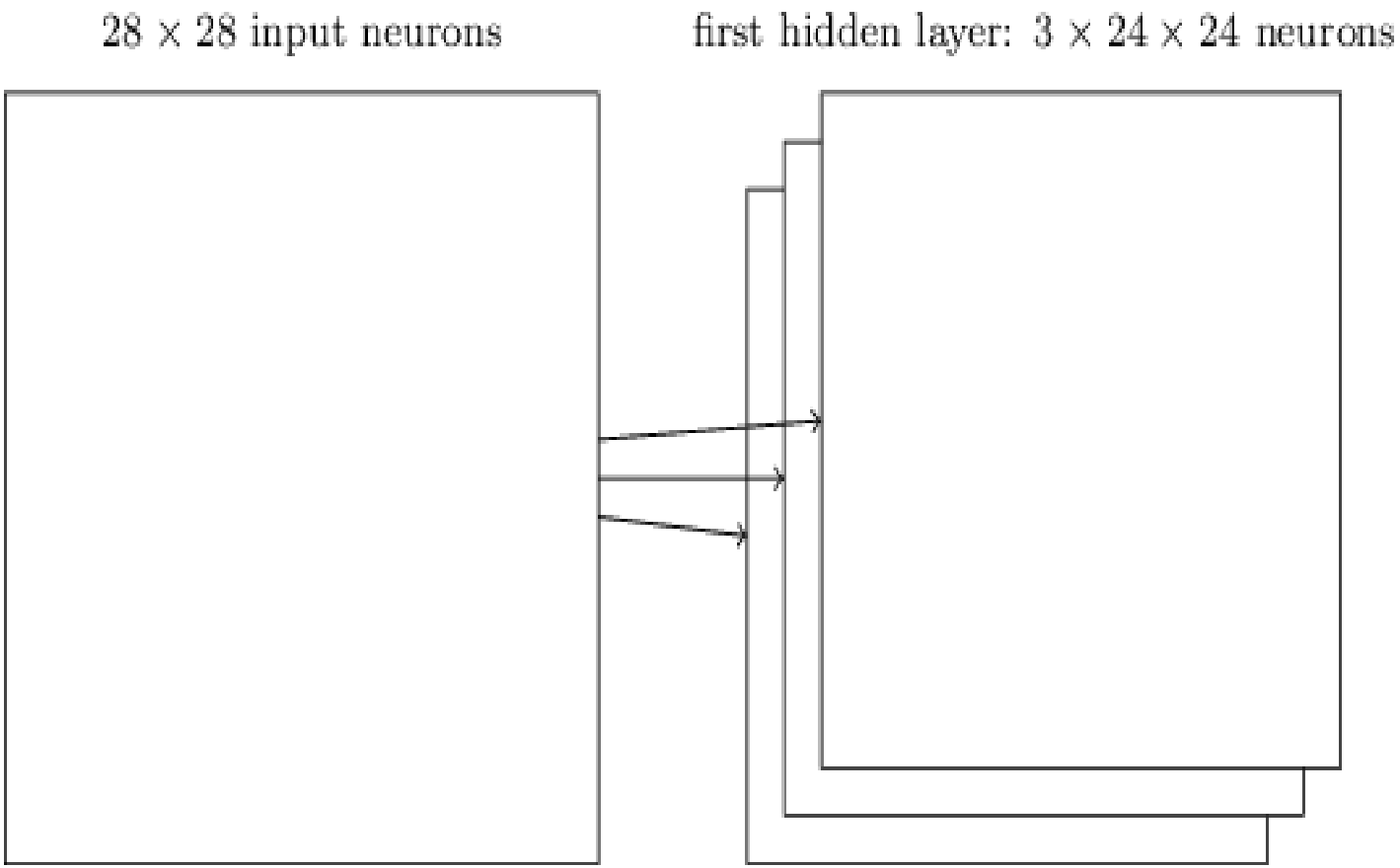


- Note: if we have a 28 ×28 input image and 5 ×5 local receptive field, then there will be 24 ×24 neurons in the hidden layer.
- Stride length in our example is 1. In fact, sometimes stride length can be more than 1.
- So stride length is a ***hyper-parameter***.

Shared weight and bias

- Each hidden neuron has a bias and 5×5 weights connected to its local receptive field.
- We use the same weight and bias for each of hidden neurons.
- This means all the neurons in the first hidden layer detect exactly the same feature, just at different locations in the input image.
- The shared weights and bias are said to define a ***filter***.
- We sometimes called the map from the input layer to the hidden layer a ***feature map***.
- To do image recognition we'll need more than one feature map. So a complete convolutional layer consists of several different feature maps.

Figure 12.14: Three feature maps



Pooling layer

- A pooling layer takes each *feature map* output from the convolutional layer and prepares a *condensed feature map*.
- One common procedure for pooling is known as *max-pooling*.

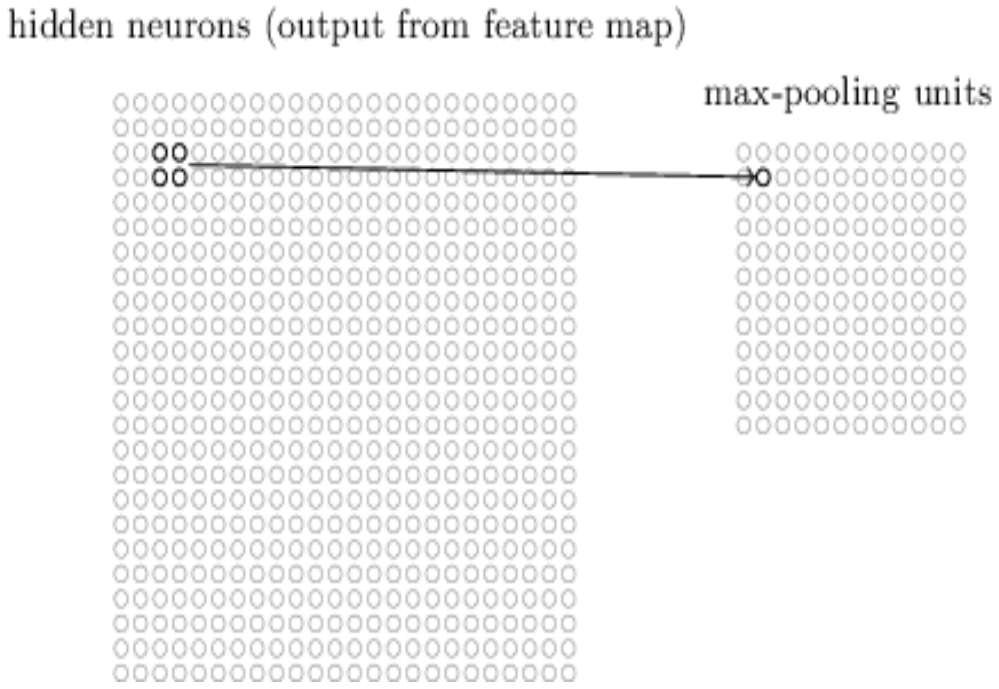
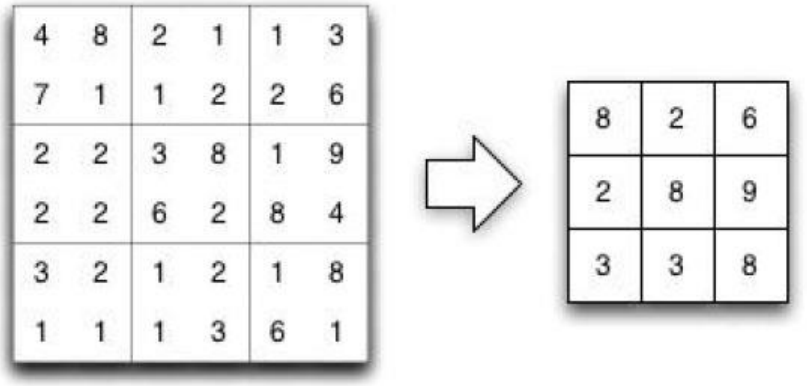


Figure 12.15



- The combined convolutional and max-pooling layers would look like:

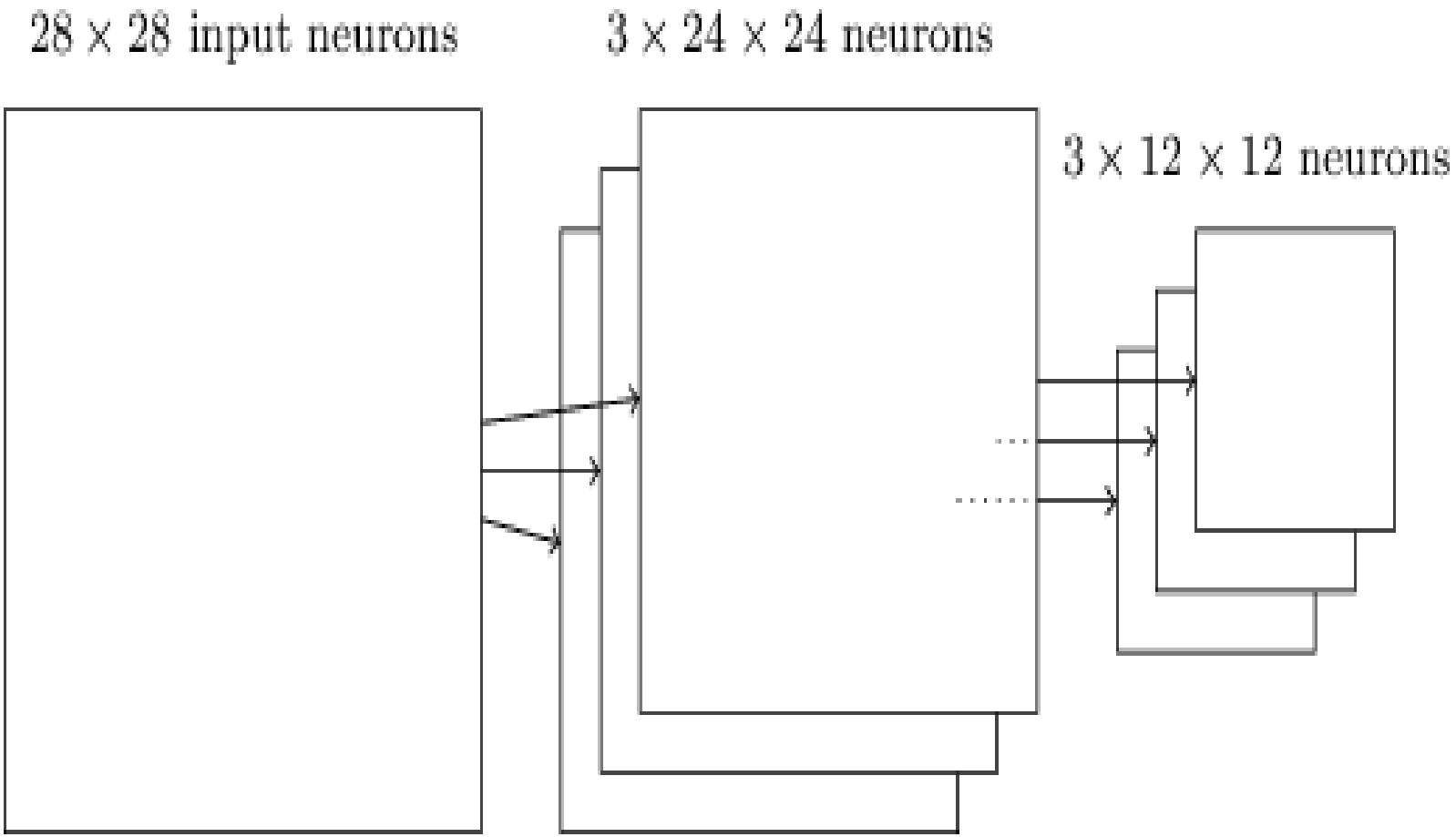


Figure 12.16

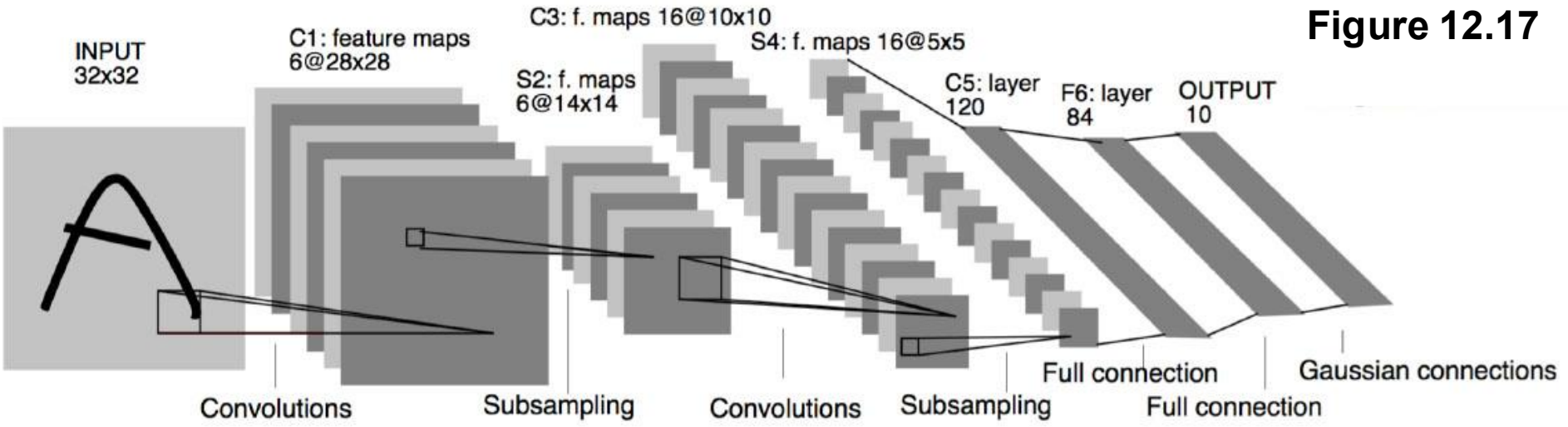


Figure: LeNET5 Architecture for handwriting recognition

Y. LeCun introduced the ***LeNET-5***, the most common type of CNNs. LeNET-5 is mainly used for classification of graphical images.

Some characteristics of LeNET5

- It uses the very common architecture: to follow each convolution layer with a subsampling layer.
- The number of filters decreases from the input to the output layer.
- The final dense layer actually performs the classification. There should be one output neuron for each class.
- We can train the LeNET5 with ***backpropagation-based*** techniques

5. Tools for Deep Neural Networks

Note: Besides DBN and CNN, there are some other well-known deep neural networks, such as Stacked AutoEncoder (SAE), Long Short Term Memory (LSTM).

One of the primary challenges of deep learning is the processing time to train a network. We have to run training algorithms for many hours, or even days, making neural network that fits well to the dataset.

We should use tools for Deep Neural Networks:

- **Keras** **TensorFlow**
- Caffe **Torch** **PyTorch**
- **Theano** (Python) [Theano directly supports GPU] http://deeplearning.net/software/theano/

Keras, Torch, Theano and **TensorFlow** include tools for both DBNN and CNN

GPU

- The *graphical processing unit* (GPU) is the part of the computer that is responsible for graphic display.
- GPU is the way that researchers solved the training problem of FFNNs.
- The graphics systems in modern video games require dedicated circuitry, which became GPU, or *video card*.
- The processing power contained in a GPU can handle mathematically intense tasks, such as neural network training.
- When applied to deep learning, the GPU performs extraordinarily well.
- However, general-purpose GPU can be difficult to use. Programs written for the GPU must employ a very low-level programming language called ***C99***. This language is very similar to the regular C language. However, in many ways, the C99 required by GPU is much more difficult than C language.
- GPU has two competing standards – CUDA and OpenCL

Architecture of GPU

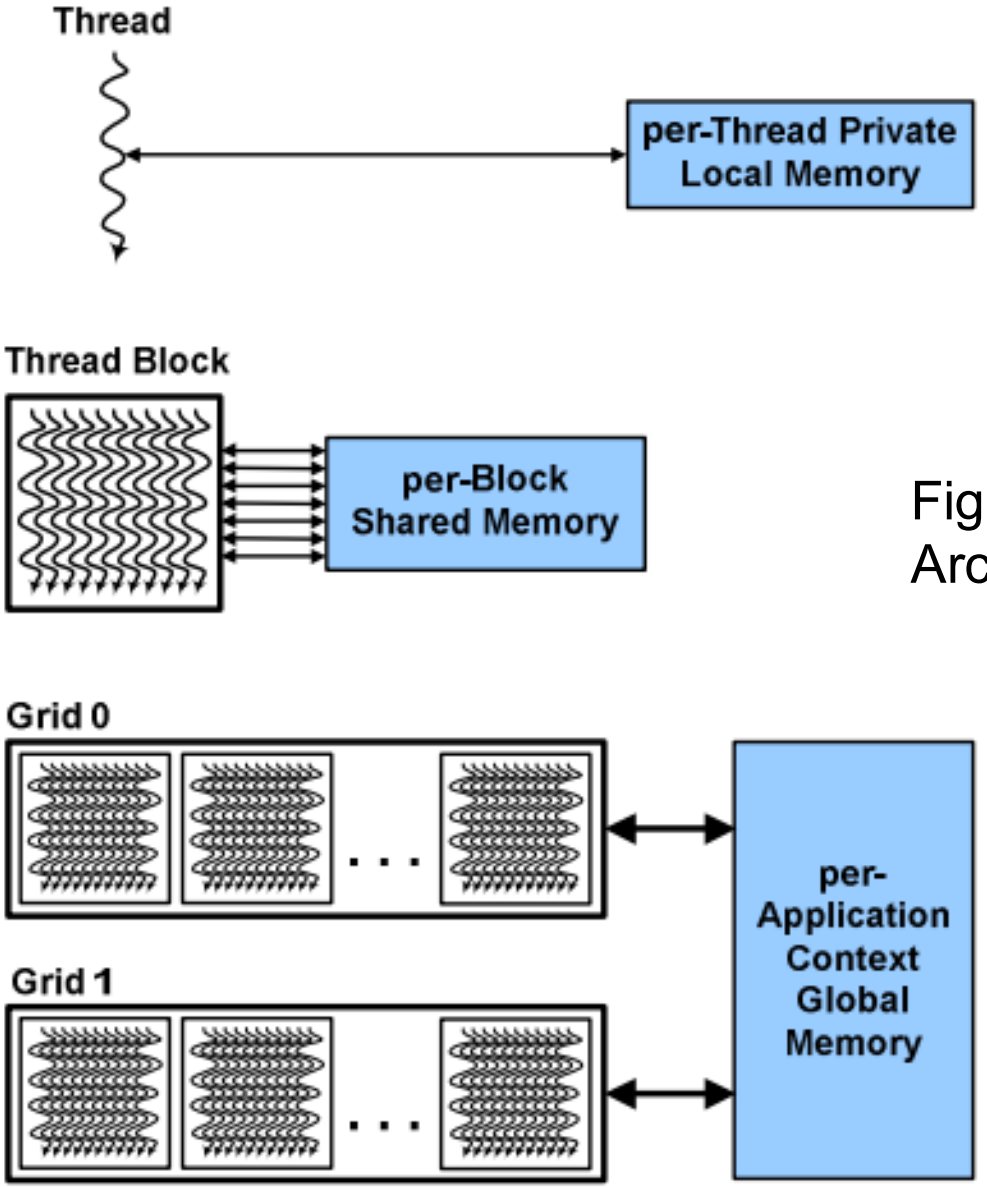


Fig. 12.17: Architecture of GPU

TensorFlow

- TensorFlow is an open-resource software library for deep learning. TensorFlow was created and is maintained by Google.
- Written by Python and C++.
- It consists of several optimization algorithms and machine learning methods.
- It can run on CPUs, GPUs, mobile devices and large scale distributed systems.
- TensorFlow programming interfaces include APIs for Python, C++ and developments for Java, GO, R, and Haskell are on the way.

http://www.tensorflow.org/

Strong and weak points of TensorFlow

Strong points

- By far, the most popular DL tool, open-source, fast evolving.
- Numerical library for *dataflow programming*.
- Efficiently works with mathematical expression involving multi-dimensional arrays.
- GPU/CPU computing, efficient in multi-GPU settings, mobile computing, high scalability.

Weak points

- Still lower level API (Application Programming Interface) difficult to use directly for creating deep learning models.
- Every computational flow must be constructed as a *static graph*.

Keras

- Keras is a Python library that provides bindings to other deep learning tools such as TensorFlow, CNTK, Theano.
- Keras can executed on GPUs and CPUs.
- Keras is developed and maintained by *Francois Chollet*, using four guiding principles:
  - User friendliness and minimalism
  - Modularity. Neural layers, cost functions, optimizers, initialization schemes, activation function, regularization schemes are all standalone modules to combine and to create new models.
  - Easy extensibility
  - Work with Python.

http://keras.io/

Strong and weak points of Keras

Strong points

- Open-source, fast evolving, with *back-end* tools from strong industrial companies like Google and Microsoft.
- Popular API for deep learning with good documentation.
- Clean and convenient way to quickly define DL models on top of back-ends (e.g. TensorFlow, Theano, CNTK).

Weak points

- Modularity and simplicity comes at the price of being less flexible. Not optimal for researching new architectures.
- *Multi-GPU* still not 100% working in terms of efficiency and easiness.

PyTorch

- PyTorch is a Python library for GPU-accelerated ML. The library is a Python interface of the same optimized C libraries that Torch uses.
- PyTorch has been developed by Facebook.
- PyTorch is written in Python, C and CUDA.
- PyTorch integrates acceleration libraries such as Intel MKL and NVIDIA (cuDNN, NCCL).
- PyTorch has become popular by allowing complex architectures to be built easily.
- PyTorch is used by both the scientific and industrial communities.

Strong and weak points of PyTorch

Strong points

- *Dynamic computational graph*
- Supports automatic differentiation for NumPy and SciPy.
- Elegant and flexible Python programming.
- Support ONNX format which allows to easily transform models between CNTK, Caffe2, PyTorch, MXNet and other Deep Learning tools.

Weak points

- Still without *mobile* solution.

6. Applications of Deep Neural Networks

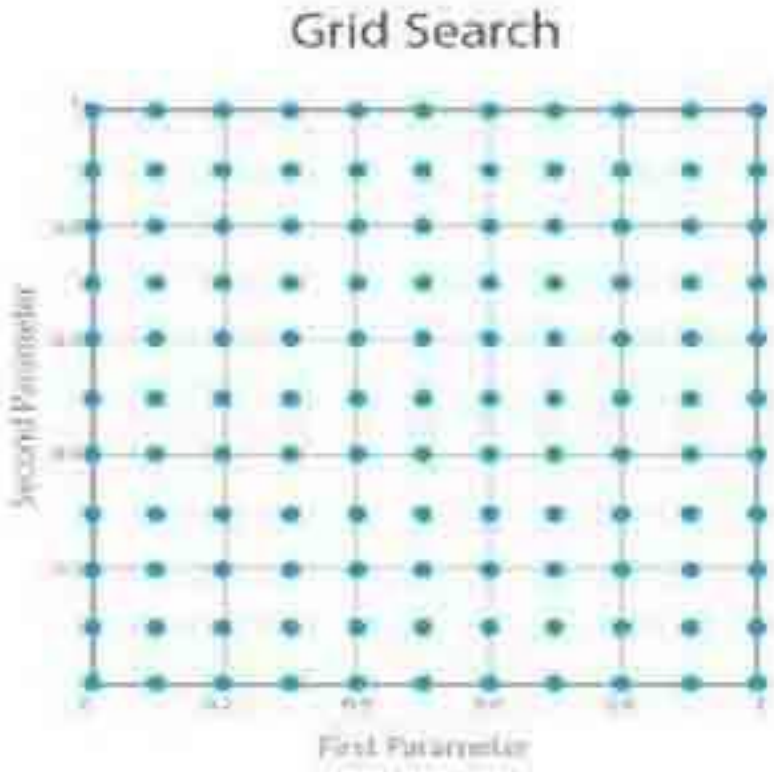
Applications:

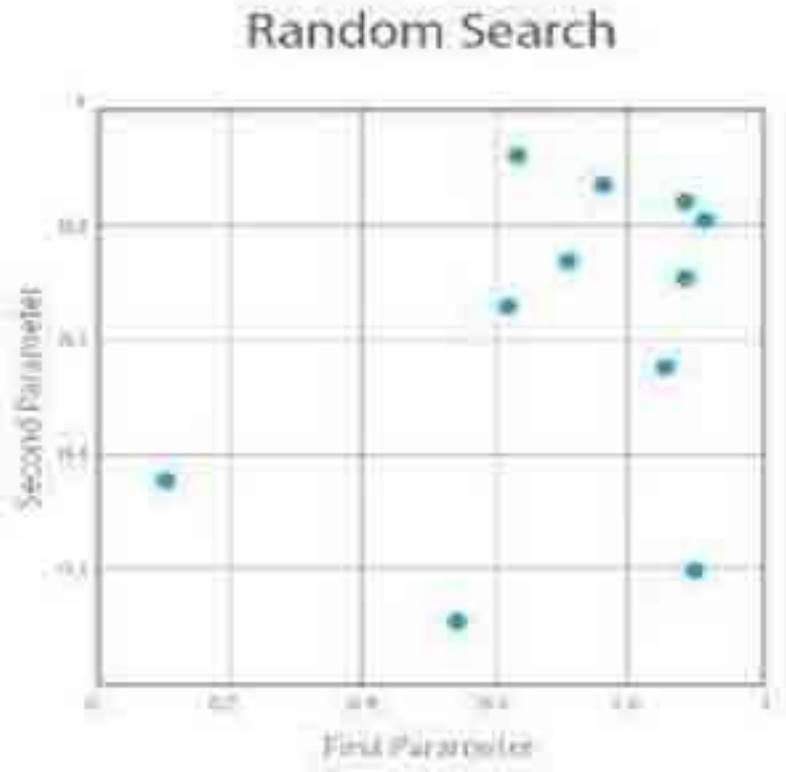
- *Speech and audio: speech recognition, audio and music processing.*
- *Image and video: image recognition, computer vision.*
- *Language modeling: machine translation, text information retrieval.*
- *Time series prediction*

7. Conclusions

- Building/learning deep architectures and hierarchies of features is highly desirable.
- Deep learning is an *emerging technology*. Despite the promising results reported so far, much need to be developed.
- The current optimization techniques for learning deep architectures should be improved.
- To make deep learning techniques scalable to very large training data, sound *parallel learning algorithms* or more effective architectures than the existing ones need to be developed.
- How to choose sensible values for *hyper-parameters* such as learning rate schedule, the number of layers and the number of units per layer, etc.

How to determine values for *hyper-parameters*





References

- J. Heaton, *Deep Learning and Neural Networks*, Heaton Research, Inc., 2015.
- I. Arel, D. C. Rose and T. P. Karmowski, *Deep Machine Learning-A new Frontier in Artificial Intelligence Research*, IEEE Computational Intelligence Magazine, November, 2010.
- N. Lopes and B. Ribeiro, *Deep Belief Networks*. In: Machine Learning for Adaptive Many-core Machines- A Practical Approach, Studies in Big Data 7, Springer, 2015.
- G.E. Hinton, S. Osindero and Y. Teh, A fast learning algorithm for deep belief nets, *Neural Computation*, vol. 14, pp.1527-1554, 2006.

Appendix: Marginal Probability

- The distinct values assumed by a discrete random variable represent mutual exclusive events.
- Similarly, for all distinct pairs of values y_1, y_2, the bivariate events ($Y_1 = y_1$, $Y_2 = y_2$), represented by (y_1, y_2), are mutual exclusive events.
- Therefore, the univariate event ($Y_1 = y_1$) is the **union** of bivariate events of the type ($Y_1=y_1$, $Y_2=y_2$), with the **union** being taken over all possible values for y_2.

Definition: Let Y_1 and Y_2 be jointly discrete random variables with probability function $p(y_1, y_2)$. Then the **marginal probability functions** of Y_1 and Y_2, respectively, are given by:

$$p_1(y_1) = \sum_{y_2} p(y_1, y_2) \qquad and \qquad p_2(y_2) = \sum_{y_1} p(y_1, y_2)$$

Terminology

- many layers: *nhiều tầng*, hierarchical architecture: *cấu trúc phân cấp*, feature detector: *bộ phát hiện đặc trưng*, simple feature: *đặc trưng đơn giản*, complex feature: *đặc trưng phức tạp*, shallow model: *mô hình không sâu*, Restricted Boltzmann Machine: *máy Boltzmann giới hạn* (RBM), encoder: *bộ mã hóa*, decoder: *bộ giải mã*, encoder-decoder paradigm: *mô thức mã hóa-giải mã*, stacked RBMs: *máy Boltzmann giới hạn xếp chồng*, generative model: *mô hình sinh*, discriminative model: *mô hình phân biệt*, visible layer: *tầng thấy được*, hidden layer: *tầng ẩn*, energy-based model: *mô hình dựa vào năng lượng*, energy function: *hàm năng lượng*, marginal probability: *xác xuất biên*, gradient ascent: *gia tăng độ dốc*, log probability: *xác xuất lấy logarithm*, positive phase: *giai đoạn thuận*, negative phase: *giai đoạn nghịch*, Gibbs sampling: *lấy mẫu Gibbs*, constrastive divergence algorithm: *giải thuật phân kỳ-tương phản*, pre-training: *tiền huấn luyện*, fine-tuning: *tinh chỉnh*, hyper-parameters: *siêu tham số*.

- convolutional neural network: *mạng nơ ron tích chập*, local receptive field: *trường tiếp nhận cục bộ*, digital filtering: *lọc số*, filter: *bộ lọc*, convolution layer: *tầng tích chập*, subsampling: *lấy mẫu giảng*, feature map: *bản đồ đặc trưng*, condensed feature map: *bản đồ đặc trưng cô đọng*, pooling: *gộp*, pooling layer: *tầng gộp*, max-pooling: *gộp cực đại*, rasterize, stride length: *độ dài trượt*, LeNET-5, dense layer: *tầng dày đặc*, graphic processing unit (GPU): *đơn vị xử lý đồ họa*.